

Standing Up the Runner

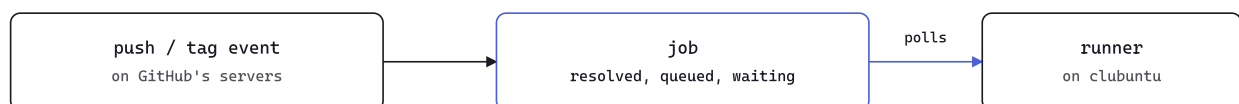
What GitHub Actions is, what every command actually did, and the real sequence of what happened getting the runner live — deviations included, not just the clean plan.

```
host: clubuntu.dala-cirius.ts.net   service: actions.runner.kat-e8-manpage.clubuntu.service
```

2026-08-16

00 What GitHub Actions actually is

A push or a tag is an event. GitHub's own servers match that event against every workflow file's trigger, resolve the matching one into a concrete **job** — an ordered list of steps, fully expanded, nothing left to interpret — and place it in a queue. A **runner** is a separate agent process, living wherever it's installed, that polls GitHub asking for queued jobs matching its labels, and executes whatever it's handed. Everything in this document is about that agent: getting one installed, identified, and running permanently on clubuntu.



this whole document is about building and installing that last box

Everything downloaded, configured, and installed below exists to make that rightmost box a real, permanently-running thing.

01 The download commands

GitHub's "New self-hosted runner" page generates a download block roughly like this:

```
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64-2.XXX.X.tar.gz -L \
```

```
https://github.com/actions/runner/releases/download/v2.XXX.X/actions-runner-linux-x64-2.XXX.X.tar xzf ./actions-runner-linux-x64-2.XXX.X.tar.gz
```

- ▶ `mkdir actions-runner && cd actions-runner` — creates a fresh subdirectory and enters it. This is the line that caused a real problem later (§07) — run from inside a directory *already* named `actions-runner`, it nests one level too deep.
- ▶ `curl -o ... -L https://github.com/actions/runner/releases/ ...` — downloads the actual runner agent: a self-contained package with the Runner.Listener and Runner.Worker binaries, a bundled Node.js (for helper scripts), and supporting files. `-L` follows redirects, since release assets are typically served from a CDN, not the URL you request directly.
- ▶ `tar xzf ...` — extracts the tarball into the current directory: `bin/`, `externals/`, `run.sh`, `config.sh`, `svc.sh`, and more.

Nothing here talks to `kat-e8/manpage` yet — this stage only unpacks generic runner software. It doesn't know which repo it belongs to until configure runs.

02 The configure command

```
./config.sh --url https://github.com/kat-e8/manpage --token <TOKEN> \  
--name clubuntu --labels clubuntu --unattended --replace
```

`config.sh` is what actually contacts GitHub and turns generic runner software into *this specific, identified* runner. It exchanges the short-lived registration token for long-lived credentials, then writes two files that matter most: `.runner` (metadata — id, name, the labels it answers to) and `.credentials` / `.credentials_rsaparams` (an RSA keypair used for all future polling — the one-hour token from GitHub's UI is spent the moment this succeeds; from then on, this keypair is what proves identity).

FLAG	WHAT IT DOES
<code>--url</code>	Which repository this runner attaches to
<code>--token</code>	One-time registration token from GitHub's UI, valid ~1 hour, spent on success
<code>--name</code> <code>clubuntu</code>	Display name in the repo's Runners list — must be unique per repo

FLAG	WHAT IT DOES
<code>--labels clubuntu</code>	Custom label added on top of the automatic <code>self-hosted</code> / <code>Linux</code> / <code>X64</code> ones — this is what <code>runs-on: [self-hosted, clubuntu]</code> actually matches against. Not optional; without it the workflows would never find this runner.
<code>--unattended</code>	Skip interactive prompts, use the flags given instead of asking
<code>--replace</code>	If a runner is already registered under this name, take over the registration instead of erroring — needed in §07 when a stale registration was left behind

03 The docker group — and a name collision worth clearing up

`usermod -aG docker ghrunner` adds `ghrunner` to the Unix `docker` group. The Docker daemon listens on `/var/run/docker.sock`, a socket owned by `root:docker` — group membership is what lets a non-root user run `docker compose build/push/pull/up` without `sudo` on every single command. It's also, by the same token, effectively root-equivalent: anything that can talk to the Docker daemon can mount the host filesystem into a container it creates. Worth knowing, not a flaw specific to this setup.

Separately — GitHub Actions has its own feature that's also called "runner groups," used at organization/enterprise level to control which repos can use which runners. This document never touches that feature; it's a Unix group, unrelated, that just happens to share a name with a different GitHub concept.

Unix docker group
ghrunner is a member
→ can read/write `/var/run/docker.sock`
→ this is what's actually used here

GitHub "runner groups"
org/enterprise access control
which repos can use which runners
→ not used anywhere in this setup

Same words, two unrelated systems — worth keeping straight since both are genuinely called "runner groups" in casual conversation.

04 What `svc.sh` actually is

`svc.sh` is bundled inside the downloaded tarball — it's not a separate GitHub tool. Its job is narrow: wrap the raw runner process as a systemd service, so it survives disconnects and

reboots instead of dying the moment a terminal closes.

SUBCOMMAND	WHAT IT DOES
<code>install <user></code>	Generates <code>/etc/systemd/system/actions.runner.<org-repo>.<name>.service</code> , with <code>User=</code> set to the given account and <code>WorkingDirectory=</code> / <code>ExecStart=</code> pointing at this exact install path. Enables it (starts on boot).
<code>start</code>	Equivalent to <code>systemctl start</code> on the generated unit
<code>status</code>	Wraps <code>systemctl status</code> for the same unit
<code>uninstall</code>	Removes the unit file, disables the service

`./run.sh` directly

tied to the terminal that launched it
closes / disconnects → runner dies

`svc.sh install + start`

runs as a systemd service
survives logout, disconnects, reboots

Both start the exact same underlying agent — only how it's supervised differs. This is why phase 4 exists at all, separate from phase 3.

05 Phase 1, what happened

Ran cleanly, exactly as planned — verified afterward via `getent passwd ghrunner` and `stat /opt/actions-runner`: the account existed with the correct home directory, correct group memberships, and the deploy directory's permissions were correctly bridged.

06 Phase 2, what happened

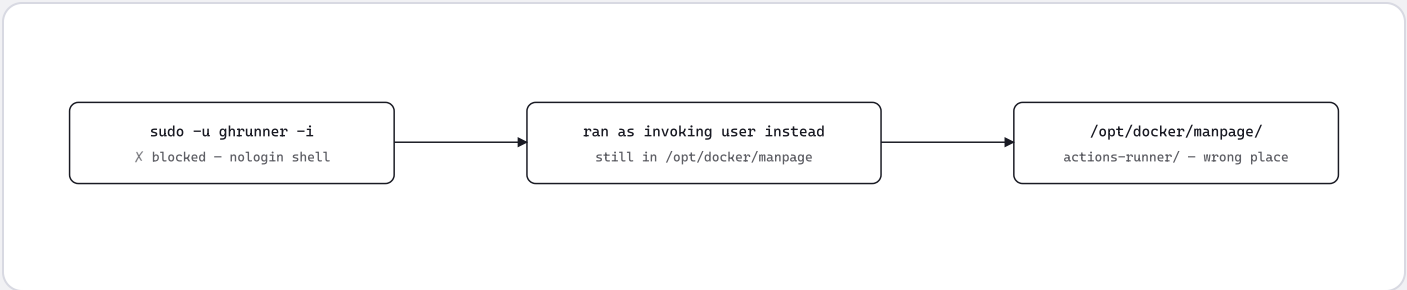
Straightforward — GitHub's UI generated a registration token. No command to log; the only artifact is the token itself, used once in phase 3 and never reused.

07 Phase 3, what actually happened

`sudo -u ghrunner -i` was the first thing tried — and it silently failed to do what was intended. `ghrunner`'s shell is `/usr/sbin/nologin` (set deliberately in phase 1, to block any interactive login as that account) — and `-i` specifically asks sudo to launch *the target*

user's own configured shell as a login shell. `nologin` refuses. The download and configure commands ended up running as the invoking user instead, from whatever directory that session happened to be in (`/opt/docker/manpage`, left over from the `.env` work) — creating a misplaced `/opt/docker/manpage/actions-runner/` rather than anything inside `/opt/actions-runner`.

That registration was real, though — running `./run.sh` manually confirmed a genuine "Listening for jobs" state, proving the token and configure step had actually succeeded against GitHub, just in the wrong location on disk.



Registration with GitHub genuinely succeeded here — the flaw was purely about where on disk it landed, discovered only in phase 4.

08 Phase 4, what actually happened

The first `svc.sh install ghrunner && svc.sh start` appeared to succeed, but `systemctl status` showed `inactive (dead)` after only 52 seconds — exited cleanly (status 0), not crashed. The likely cause: `.credentials` had been created by the invoking user in phase 3, not `ghrunner`; when the service was configured to run as `ghrunner`, it couldn't read its own credential files and gave up quietly.

Correcting this meant unregistering, deleting the misplaced directory, and redoing phases 3–4 properly — which surfaced three more small obstacles in sequence, each with its own real cause:

OBSTACLE	ACTUAL CAUSE	FIX
<code>sudo -u ghrunner -i</code> again	Same nologin block as phase 3	<code>sudo -u ghrunner bash</code> — names a shell explicitly instead of asking sudo to resolve the account's own (blocked) one
GitHub's own <code>mkdir actions-runner</code>	Run from inside a directory already named <code>actions-runner</code> (ghrunner's home), nesting one level deep	<code>shopt -s dotglob; mv .../actions-runner/* ... ; rmdir ...</code> — flatten, including hidden <code>.runner</code> / <code>.credentials</code>

OBSTACLE	ACTUAL CAUSE	FIX
"runner exists with the same name clubuntu"	The phase-3 registration was likely never cleanly removed (its own removal command also used the blocked <code>-u ghrunner -i</code>)	<code>--replace</code> on <code>config.sh</code> — safely takes over the existing name
<code>cd /opt/actions-runner</code> as katlego	Correct, intentional — that directory is <code>0750 ghrunner:ghrunner</code> , and katlego was never bridged into <i>that</i> group (only the reverse)	<code>sudo -i</code> — a root shell bypasses the permission wall entirely
<code>sudo cd ...</code> → command not found	<code>cd</code> is a shell builtin, not an executable — there's nothing for <code>sudo</code> to run	Not something to fix, just to know — became moot once already root via <code>sudo -i</code>

With the directory flattened and a clean registration owned entirely by `ghrunner`, `svc.sh install ghrunner && svc.sh start` produced a genuinely persistent service this time.

09 Every command run on clubuntu, in order

#	COMMAND	WHY
1	<code>useradd --system --create-home --home-dir /opt/actions-runner --shell /usr/sbin/nologin ghrunner</code>	Dedicated, non-login account for the runner
2	<code>usermod -aG docker ghrunner</code>	Lets it drive the Docker daemon
3	<code>usermod -aG katlego ghrunner</code>	Bridges write access to <code>/opt/docker/manpage</code>
4	<code>chmod -R g+rwX /opt/docker/manpage</code> + <code>chmod g+s ...</code>	Group-writable, with new files inheriting the group automatically
5	<code>sudo -u ghrunner -i</code> (<i>failed</i>)	Attempted to get a ghrunner shell — blocked by nologin
6	download + <code>config.sh</code> (<i>as wrong user</i>)	Registered successfully, but in the wrong location
7	<code>./run.sh</code>	Manually confirmed "Listening for jobs" — proved registration worked
8	<code>svc.sh install ghrunner</code> + <code>start</code> (<i>first attempt</i>)	Installed but exited after 52s — credential ownership mismatch
9	<code>svc.sh stop</code> + <code>uninstall</code>	Tear down the broken service
10	<code>config.sh remove --token ...</code>	Unregister the misplaced runner from GitHub
11	<code>rm -rf /opt/docker/manpage/actions-runner</code>	Delete the misplaced install (siblings <code>.env</code> / <code>db/</code> / <code>xlsx</code> untouched)
12	<code>sudo -u ghrunner bash</code> + <code>cd /opt/actions-runner</code> + <code>pwd</code>	Correct shell workaround, with a checkpoint before proceeding

#	COMMAND	WHY
13	<code>download + config.sh --replace</code>	Reinstall correctly, taking over the stale name
14	<code>sudo -i + cd /opt/actions-runner</code>	Root shell to get past the (correct) permission wall
15	<code>ls -la</code>	Diagnosed the nested <code>actions-runner/</code> subdirectory
16	<code>shopt -s dotglob; mv ... ; rmdir ...</code>	Flattened the structure, including hidden credential files
17	<code>svc.sh install ghrunner + start + status</code>	Genuinely persistent this time — confirmed <code>active (running)</code>

10 Final verified state

```
Active: active (running)
Main PID: runsvc.sh
├─ node ./bin/RunnerService.js
├─ Runner.Listener run --startuptype service
└─ Runner.Worker spawnclient ... (a job was already
executing)
```

The Worker process suggests it picked up the `build` job that had been queued since the very first push to `main`, this morning, with no runner yet to claim it.

```
git-docs - 8 · host: clubuntu.dala-cirius.ts.net · service: actions.runner.kat-e8-
manpage.clubuntu.service
```